



HiveMind

Multi-agent reinforcement learning and emergent communication
for cooperative warehouse robots

Can a robot team learn to cooperate, and to talk?

The problem

- Warehouses use fleets of robots to fetch items and bring them to a drop-off point.
- Robots share narrow aisles, compete for the same items and block each other.
- Hand-coded coordination breaks when the layout changes.

Metrics

Success rate · makespan (steps to finish) · collisions

Goal 1 Cooperative MARL

4 robots learn to find, pick up and deliver 12 cartons to a depot as a team. No map is given, the layout is randomised every episode, and obstacles are sensed only through LiDAR.

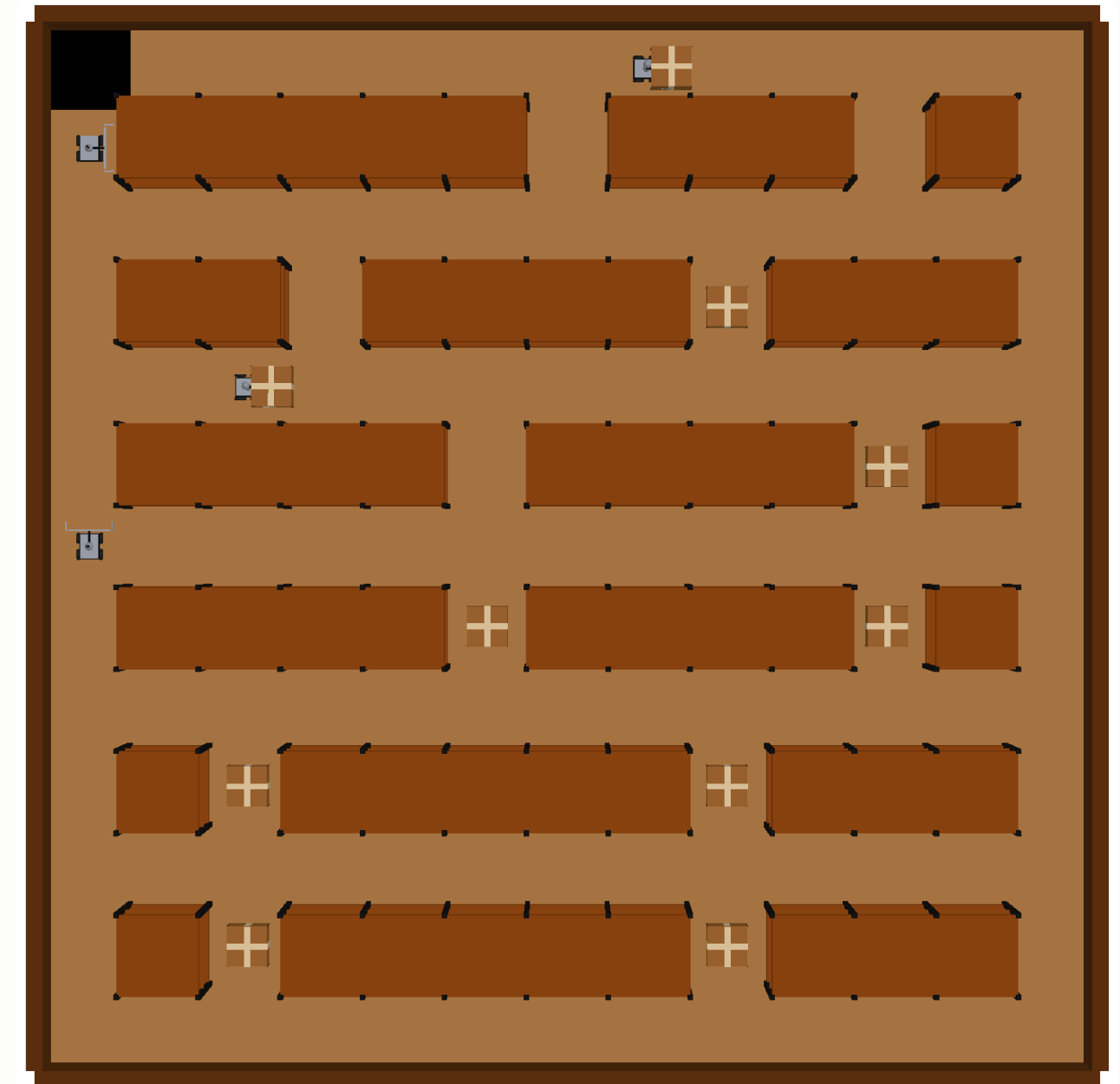
Goal 2 Emergent communication

Give the robots a discrete message channel and study whether they learn to use it. Ablation: **no comm** vs **comm**.

ENVIRONMENT

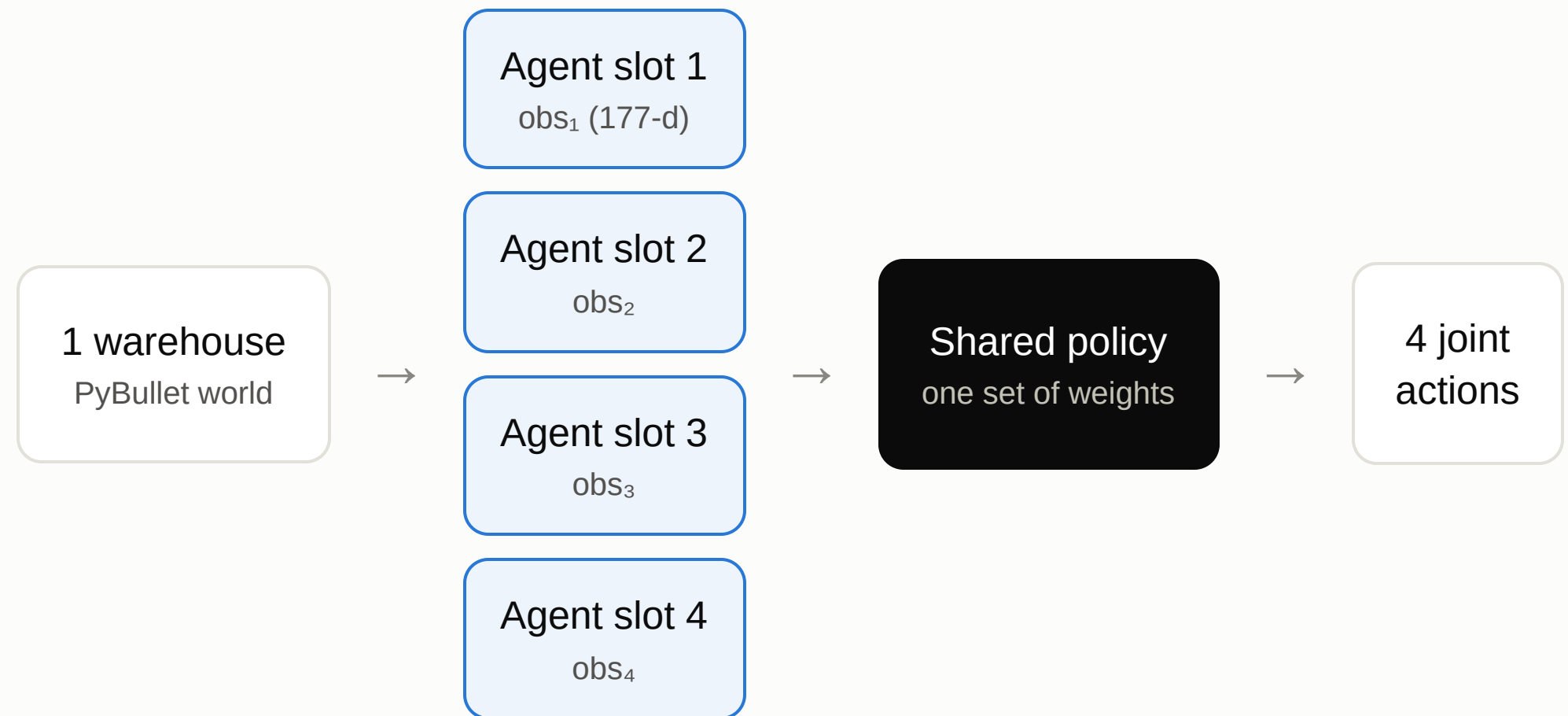
A simulated warehouse in PyBullet

- **Simulator:** PyBullet rigid-body physics, wrapped as a Gymnasium environment.
- **World:** 13×13 grid, 6 shelf rows with randomised gaps each episode, 12 cartons, corner depot.
- **Agents:** 4 differential-drive robots (URDF) with a gripper arm and a 2-D LiDAR (72 rays, 270° , noisy).
- **Actions:** forward, backward, turn left/right, pick up, drop off, stay.
- **Partial observability:** no map; obstacles are visible only within LiDAR range.
- **Termination:** all cartons delivered, or the step cap.



MARL formulation: one brain, four robots

- **Cooperative Dec-POMDP:** a shared team objective, with each agent acting on its own local observation.
- **Parameter sharing:** one policy network for all 4 agents; the agents stay distinct through their own observations.
- **Decentralised execution:** each robot runs the shared network on its own input.
- **Bridge to single-agent RL:** a custom Stable-Baselines3 `VecEnv` presents each 4-robot world as 4 agent slots.
- **Critic:** decentralised (per-agent input). A centralised MAPPO-style critic is the next upgrade.

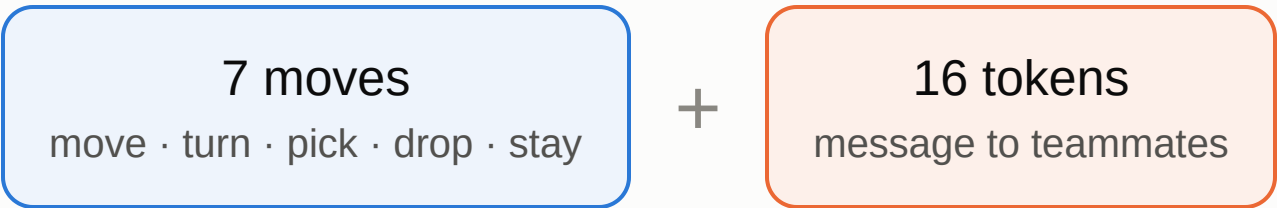


Observation and action spaces

Observation: a fixed 177-d vector per agent

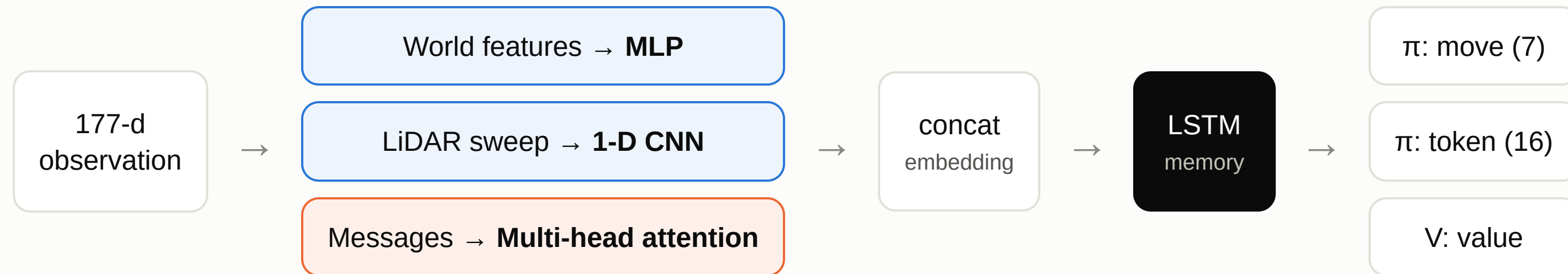
BLOCK	CONTENTS
Ego state	pose, velocity, carrying flag
Teammates	poses and carrying flags
Task state	carton status and positions, depot offset, elapsed time
LiDAR	72 normalised range readings
Messages	3 teammates × 16-token one-hot (48)

Action: MultiDiscrete([7, 16])



Fixed observation width. Message slots are reserved in both arms, so the comm and no-comm models have identical input/output shapes, which keeps the ablation clean.

Network and algorithm: Recurrent PPO



Custom feature extractor (PyTorch)

- **1-D CNN** (strided convolutions) exploits the angular structure of the LiDAR sweep.
- **Self-attention** over the 3 senders, max-pooled so sender order doesn't matter.

Recurrent PPO (sb3-contrib)

- **LSTM** gives memory under partial observability.
- Clipped surrogate objective, **GAE** advantages, entropy bonus.
- **Separate actor and critic extractors**, so value gradients don't interfere with the policy.

Emergent communication

- **Channel:** 16 discrete tokens, broadcast every step. Each robot hears the other 3; teammates receive a token the next step.
- **Emergent:** no meaning is assigned to any token. Whether to use the channel, and what tokens mean, must be learned purely from the team reward.
- **Receiver:** multi-head attention over incoming messages.

Ablation design: one variable changed

	NO COMM	COMM
Tokens produced	yes	yes
Tokens delivered	no (slots held at 0)	yes
Network / action space	same	same
Reward / hyperparameters	same	same

Tuning only the comm run would confound the comparison, so hyperparameters stay fixed.

Reward design and training

Reward = $0.8 \times \text{shared} + 0.2 \times \text{individual}$

- **Shared:** per-delivery reward, completion bonus, makespan bonus, time penalty, collision cost.
- **Individual:** own pick-ups and deliveries, invalid actions, idling.
- **Potential-based shaping** (Ng et al., 1999): $F = \Phi(s') - \Phi(s)$, $\Phi = -$ (work remaining). It telescopes, so it can't be farmed and doesn't change which policy is optimal.
- **Credit assignment:** asymmetric collision blame; only the agent that moved takes the penalty.

Training setup

- **Curriculum:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 12$ cartons, promoted on rolling success rate.
- **Parallel simulation:** multi-process `SubprocVecEnv`, 8 worlds = 32 agent streams.
- `VecNormalize` reward normalisation; $\gamma = 0.999$.

Tech stack

Python · PyBullet · Gymnasium · PyTorch · Stable-Baselines3 / sb3-contrib · TensorBoard

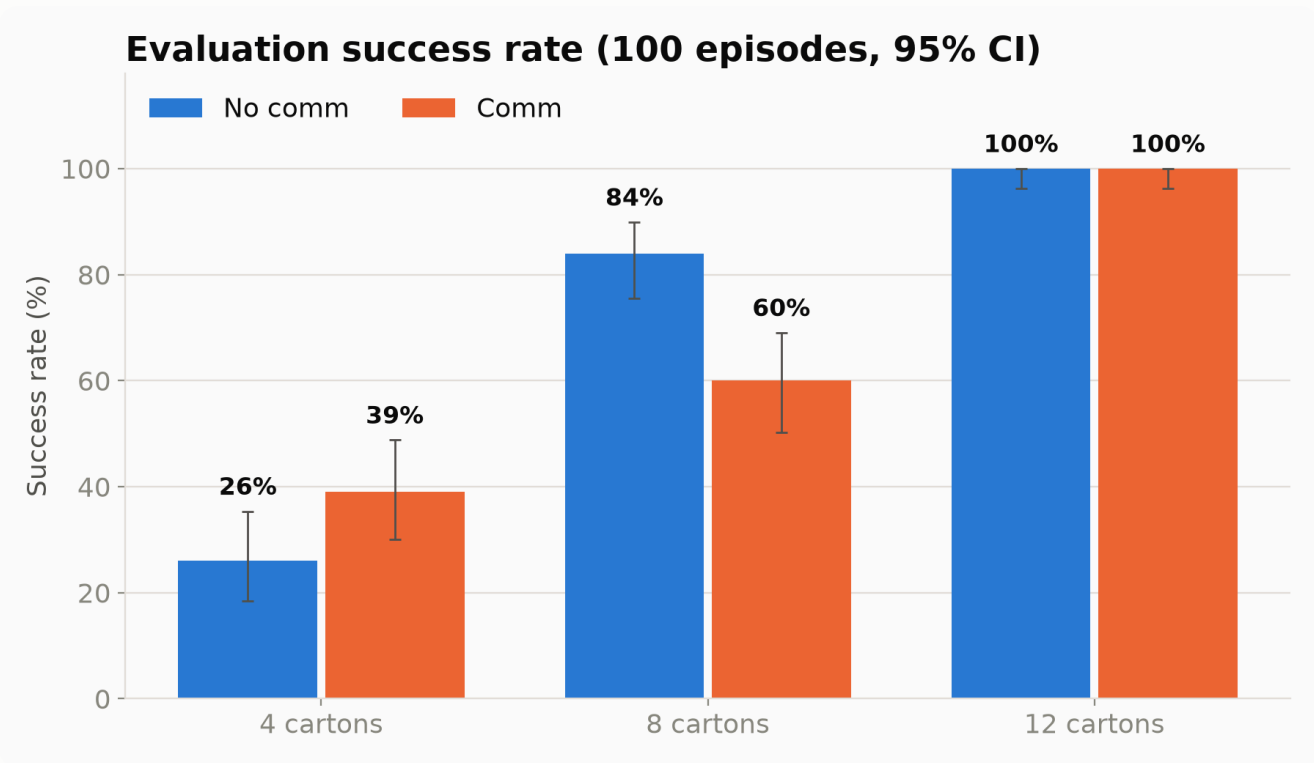
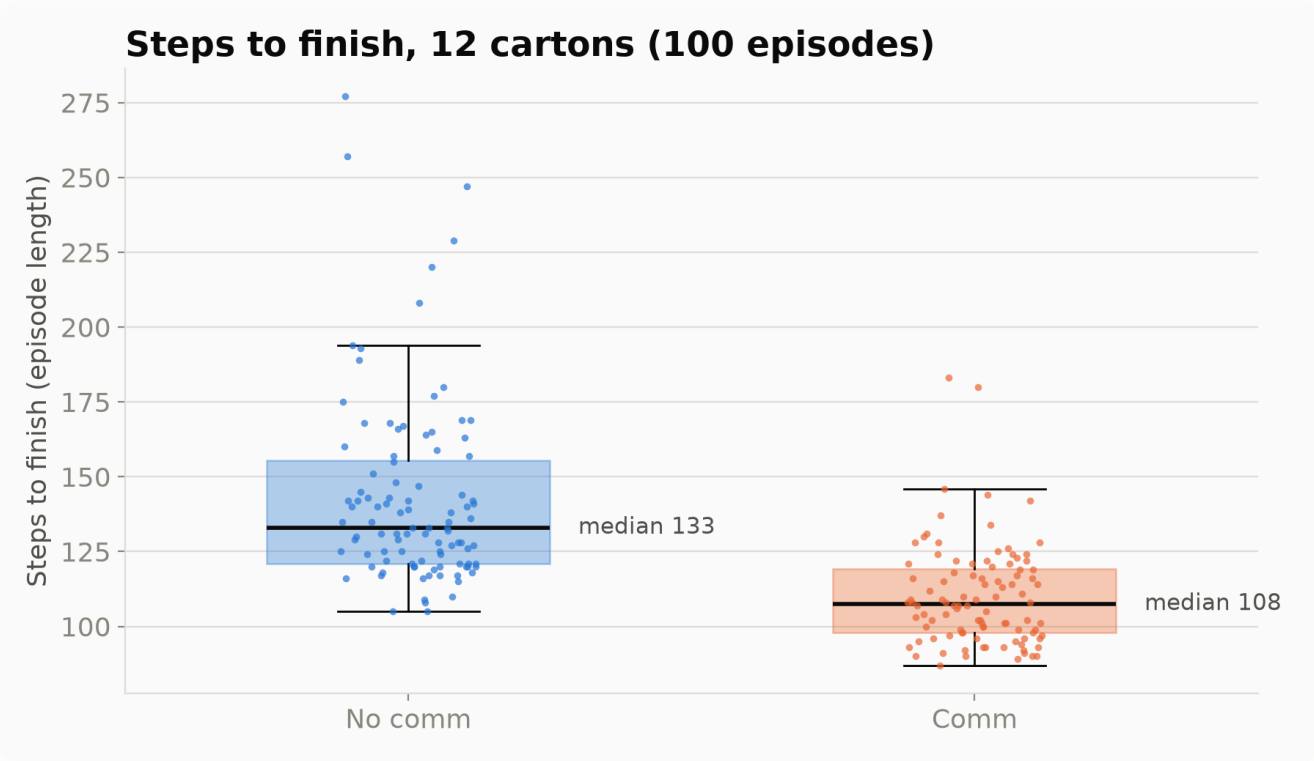
Training: both arms learn the full task

Graph: training success rate, comm vs no comm
t1_success_rate_compare.png

Graph: episode length, comm vs no comm
t2_episode_length_compare.png

- **Same curriculum, both converge:** 12 cartons reached at ~2.2 M (comm) and ~2.6 M (no comm) steps; training success → ~100%.
- **Critic converged:** explained variance ≈ 0.999 , value loss → ~0.001.
- **Budgets differ:** no comm 13.5 M steps, comm 30 M steps (19.2 h). At equal steps (~13.5 M), episode length is about the same (~200); comm kept improving to ~123 by 30 M.
- **PPO diagnostics:** approx. KL peaked ~0.09 and clip fraction ~0.39 mid-run. Both decay to 0 as the linear LR schedule reaches 0; the late rise in policy loss is expected, not divergence.

Evaluation: no comm vs comm

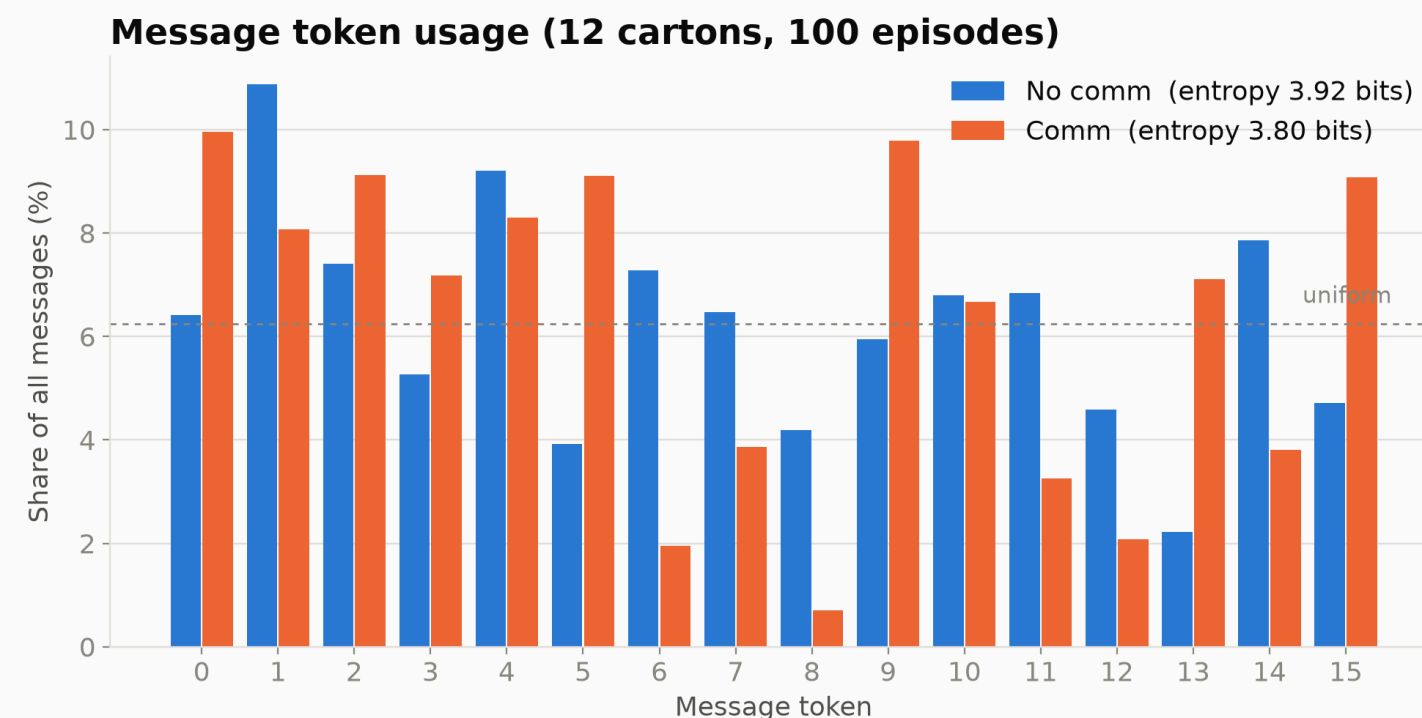
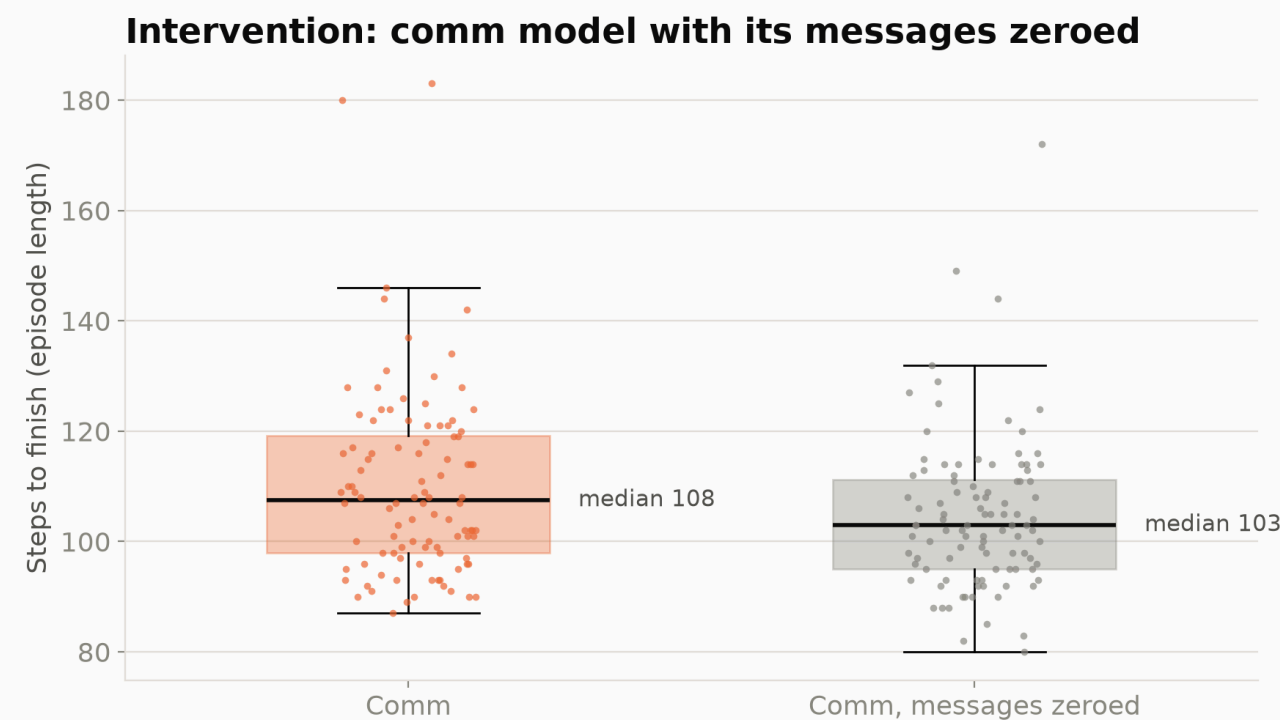


100 headless episodes per setting · deterministic policy · identical warehouse layouts for both models

12 CARTONS	NO COMM	COMM
Success rate	100%	100%
Steps (mean / median)	143 / 133	110 / 108
Collisions / episode	9.4	6.8
Invalid actions / episode	93	33

- **Full task:** comm finishes ~23% faster, with ~28% fewer collisions.
- **Fewer cartons:** both weaker (curriculum overfitting). Comm is better at 4 cartons (39% vs 26%); no comm is better at 8 (84% vs 60%).

Did communication emerge?



- **Intervention test:** zeroing the comm model's incoming messages still gives 100% success, 105 vs 110 steps, so **no loss**.
- **Token statistics:** entropy 3.80 of a maximum 4 bits (near uniform); mutual information with "carrying" is only 0.013 bits.
- **Conclusion:** the channel is open, but the policy doesn't rely on it. The gain over no comm comes from longer training (30 M vs 13.5 M), not from messages.

What we learned

- An open channel alone doesn't produce communication: teammate poses are already observed, so messages add little.
- Measuring emergent communication needs causal tests (interventions), not just performance.
- Next: hide teammate information, add a listener or information bonus, use a centralised critic, match training budgets.



Thank you

Questions?